7. Useful References

1. Online Resources

- **FreeCodeCamp**

  Website: freecodecamp.org

  YouTube Channel: FreeCodeCamp

  Popular Videos:

  - Intro to Machine Learning
  - Machine Learning Crash Course

- **DataCamp**

  Website: datacamp.com

2. Dataset

- **Kaggle**: kaggle.com/datasets (huge collection of real datasets)
- **UCI ML Repository**: archive.ics.uci.edu/ml (classic ML datasets)
- **Google Dataset Search**: datasetsearch.research.google.com
- **Government Data**: data.gov (US), data.gov.sg (Singapore)
- **Awesome Public Datasets**: github.com/awesomedata/awesome-public-datasets

🎯 Let's Code: YouTube Trending Video Analysis

In this notebook, we'll walk through a complete mini data science project using **real-world YouTube trending data**.

What We'll Cover:

1. **Data Ingestion** – Load trending video data from CSV and SQL database
2. **Data Preprocessing** – Remove invalid, zero, or error-filled entries
3. **Data Exploration** – Visualize distributions and trends in views, dates, and features

4. **Feature Engineering** – Create new variables like ratios, title length, and publish timing
5. **Model Selection** – Compare classification models (Random Forest, Gradient Boosting, AdaBoost)
6. **Hyperparameter Tuning** – Use GridSearchCV to find the best model settings
7. **Model Training & Evaluation** – Train models and evaluate performance using accuracy
8. **Model Deployment** – Launch a Gradio dashboard to showcase model results and visual insights

We'll also use:

- **SQLite** for data handling
- **Pandas/Matplotlib/Seaborn** for analysis
- **Scikit-learn** for machine learning
- **Gradio** to create a simple interactive interface

Pipeline 1 - Data Ingestion

Step 1: Install Dependencies

Install specific versions of essential libraries for data handling, visualization, modeling, and web UI (Gradio).

In [ ]:
```
# Cell 1: Install specific library versions
!pip install pandas==1.5.3 \
                numpy==1.24.2 \
                sqlalchemy==2.0.8 \
                scikit-learn==1.2.2 \
                matplotlib==3.7.1 \
                seaborn==0.12.2 \
                gradio==4.44.1
```

```
Requirement already satisfied: pandas==1.5.3 in /usr/local/lib/python3.11/di
st-packages (1.5.3)
Requirement already satisfied: numpy==1.24.2 in /usr/local/lib/python3.11/di
st-packages (1.24.2)
Requirement already satisfied: sqlalchemy==2.0.8 in /usr/local/lib/python3.1
1/dist-packages (2.0.8)
Requirement already satisfied: scikit-learn==1.2.2 in /usr/local/lib/python
3.11/dist-packages (1.2.2)
Requirement already satisfied: matplotlib==3.7.1 in /usr/local/lib/python3.1
1/dist-packages (3.7.1)
Requirement already satisfied: seaborn==0.12.2 in /usr/local/lib/python3.11/
dist-packages (0.12.2)
Requirement already satisfied: gradio==4.44.1 in /usr/local/lib/python3.11/d
ist-packages (4.44.1)
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/pyth
on3.11/dist-packages (from pandas==1.5.3) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dis
t-packages (from pandas==1.5.3) (2025.2)
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/py
thon3.11/dist-packages (from sqlalchemy==2.0.8) (4.14.0)
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.1
1/dist-packages (from sqlalchemy==2.0.8) (3.2.3)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.11/dis
t-packages (from scikit-learn==1.2.2) (1.15.3)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.11/di
st-packages (from scikit-learn==1.2.2) (1.5.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python
3.11/dist-packages (from scikit-learn==1.2.2) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.1
1/dist-packages (from matplotlib==3.7.1) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dis
t-packages (from matplotlib==3.7.1) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.1
1/dist-packages (from matplotlib==3.7.1) (4.58.4)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.1
1/dist-packages (from matplotlib==3.7.1) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/
dist-packages (from matplotlib==3.7.1) (24.2)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.11/di
st-packages (from matplotlib==3.7.1) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.1
1/dist-packages (from matplotlib==3.7.1) (3.2.3)
Requirement already satisfied: aiofiles<24.0,>=22.0 in /usr/local/lib/python
3.11/dist-packages (from gradio==4.44.1) (23.2.1)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.11/
dist-packages (from gradio==4.44.1) (4.9.0)
Requirement already satisfied: fastapi<1.0 in /usr/local/lib/python3.11/dist
-packages (from gradio==4.44.1) (0.115.12)
Requirement already satisfied: ffmpy in /usr/local/lib/python3.11/dist-packa
ges (from gradio==4.44.1) (0.6.0)
Requirement already satisfied: gradio-client==1.3.0 in /usr/local/lib/python
3.11/dist-packages (from gradio==4.44.1) (1.3.0)
Requirement already satisfied: httpx>=0.24.1 in /usr/local/lib/python3.11/di
st-packages (from gradio==4.44.1) (0.28.1)
Requirement already satisfied: huggingface-hub>=0.19.3 in /usr/local/lib/pyt
hon3.11/dist-packages (from gradio==4.44.1) (0.33.0)
```

```
Requirement already satisfied: importlib-resources<7.0,>=1.3 in /usr/local/l
ib/python3.11/dist-packages (from gradio==4.44.1) (6.5.2)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.11/dist-
packages (from gradio==4.44.1) (3.1.6)
Requirement already satisfied: markupsafe~=2.0 in /usr/local/lib/python3.11/
dist-packages (from gradio==4.44.1) (2.1.5)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.11/dist
-packages (from gradio==4.44.1) (3.10.18)
Requirement already satisfied: pydantic>=2.0 in /usr/local/lib/python3.11/di
st-packages (from gradio==4.44.1) (2.11.7)
Requirement already satisfied: pydub in /usr/local/lib/python3.11/dist-packa
ges (from gradio==4.44.1) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.9 in /usr/local/lib/pyt
hon3.11/dist-packages (from gradio==4.44.1) (0.0.20)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.1
1/dist-packages (from gradio==4.44.1) (6.0.2)
Requirement already satisfied: ruff>=0.2.2 in /usr/local/lib/python3.11/dist
-packages (from gradio==4.44.1) (0.11.13)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/pytho
n3.11/dist-packages (from gradio==4.44.1) (2.10.0)
Requirement already satisfied: tomlkit==0.12.0 in /usr/local/lib/python3.11/
dist-packages (from gradio==4.44.1) (0.12.0)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.1
1/dist-packages (from gradio==4.44.1) (0.16.0)
Requirement already satisfied: urllib3~=2.0 in /usr/local/lib/python3.11/dis
t-packages (from gradio==4.44.1) (2.4.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.11/
dist-packages (from gradio==4.44.1) (0.34.3)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-pack
ages (from gradio-client==1.3.0->gradio==4.44.1) (2025.3.2)
Requirement already satisfied: websockets<13.0,>=10.0 in /usr/local/lib/pyth
on3.11/dist-packages (from gradio-client==1.3.0->gradio==4.44.1) (12.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.11/dist-p
ackages (from anyio<5.0,>=3.0->gradio==4.44.1) (3.10)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dis
t-packages (from anyio<5.0,>=3.0->gradio==4.44.1) (1.3.1)
Requirement already satisfied: starlette<0.47.0,>=0.40.0 in /usr/local/lib/p
ython3.11/dist-packages (from fastapi<1.0->gradio==4.44.1) (0.46.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-pac
kages (from httpx>=0.24.1->gradio==4.44.1) (2025.6.15)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/di
st-packages (from httpx>=0.24.1->gradio==4.44.1) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.11/dist-p
ackages (from httpcore==1.*->httpx>=0.24.1->gradio==4.44.1) (0.16.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-pa
ckages (from huggingface-hub>=0.19.3->gradio==4.44.1) (3.18.0)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-pa
ckages (from huggingface-hub>=0.19.3->gradio==4.44.1) (2.32.3)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.11/dis
t-packages (from huggingface-hub>=0.19.3->gradio==4.44.1) (4.67.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.2 in /usr/local/lib/python
3.11/dist-packages (from huggingface-hub>=0.19.3->gradio==4.44.1) (1.1.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/pyth
on3.11/dist-packages (from pydantic>=2.0->gradio==4.44.1) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/pytho
n3.11/dist-packages (from pydantic>=2.0->gradio==4.44.1) (2.33.2)
```

```
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/py
thon3.11/dist-packages (from pydantic>=2.0->gradio==4.44.1) (0.4.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-pa
ckages (from python-dateutil>=2.8.1->pandas==1.5.3) (1.17.0)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.11/dis
t-packages (from typer<1.0,>=0.12->gradio==4.44.1) (8.2.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.
11/dist-packages (from typer<1.0,>=0.12->gradio==4.44.1) (1.5.4)
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.11/di
st-packages (from typer<1.0,>=0.12->gradio==4.44.1) (13.9.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/pytho
n3.11/dist-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio==4.44.1)
(3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pyt
hon3.11/dist-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio==4.44.1)
(2.19.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/py
thon3.11/dist-packages (from requests->huggingface-hub>=0.19.3->gradio==4.4
4.1) (3.4.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-
packages (from markdown-it-py>=2.2.0->rich>=10.11.0->typer<1.0,>=0.12->gradi
o==4.44.1) (0.1.2)
```

Step 2: Import Libraries & Set Plot Style

Import core libraries for data manipulation (pandas, numpy), SQL handling (sqlite3, sqlalchemy), machine learning (sklearn), plotting (matplotlib, seaborn), and dashboarding (gradio). Also sets a clean and readable default plotting style using Matplotlib.

In [ ]:
```python
# Cell 2: Imports & styling
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sqlite3
from sqlalchemy import create_engine

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB, BernoulliNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA,
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif
from sklearn.metrics import classification_report, roc_auc_score, roc_curve,

import gradio as gr
import seaborn as sns

# Use default matplotlib style for simplicity
plt.style.use('default')
```

Step 3: Load Dataset & Initial Preview

Load the raw YouTube dataset (USvideos.csv) and display its dimensions and a sample of the first few rows to get an initial sense of the data.

In [ ]:
```
# Cell 3: Data Ingestion & preview
raw_df = pd.read_csv('USvideos.csv')
print("Raw data shape:", raw_df.shape)
raw_df.head(3)
```

```
Raw data shape: (40949, 16)
```

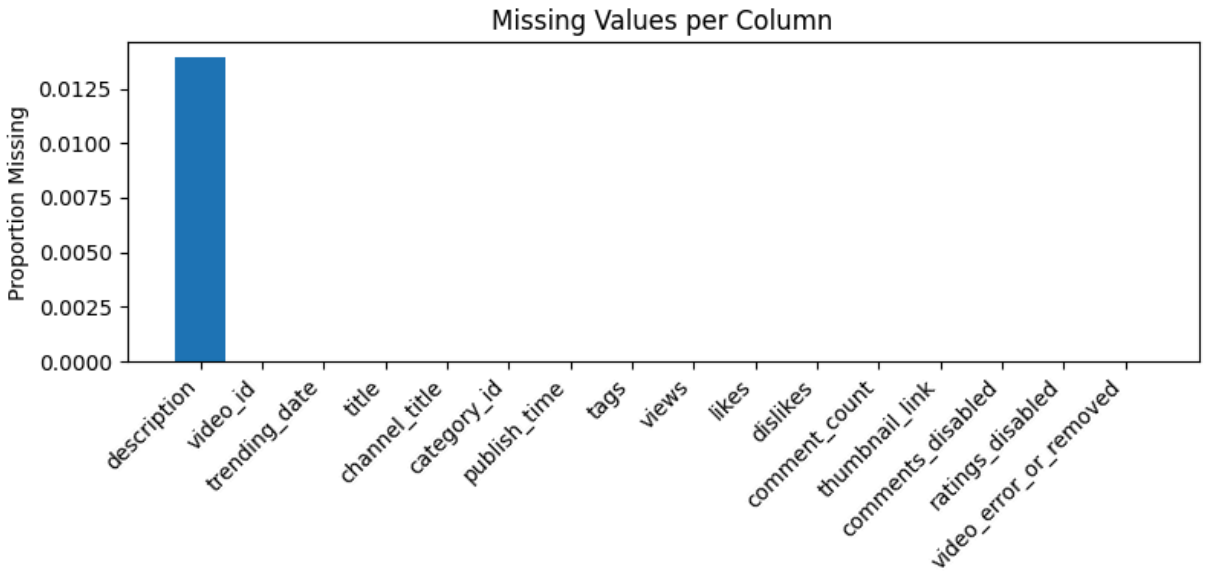
Out[ ]:

	video_id	trending_date	title	channel_title	category_id	pu
0	2kyS6SvSYSE	17.14.11	WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	22	13T17:
1	1ZAPwfrtAFY	17.14.11	The Trump Presidency: Last Week Tonight with J...	LastWeekTonight	24	13T07:
2	5qpjK5DgCt4	17.14.11	Racist Superman | Rudy Mancuso, King Bach & Le...	Rudy Mancuso	23	12T19:

Step 4: Visualize Missing Data Proportions

Calculate and visualize the proportion of missing values per column to understand data quality and decide if any preprocessing or filtering is needed.

In [ ]:
```
# Cell 4: Missing-value proportions (simple bar chart)
missing = raw_df.isnull().mean().sort_values(ascending=False)
plt.figure(figsize=(8,4))
plt.bar(missing.index, missing.values)
plt.xticks(rotation=45, ha='right')
plt.ylabel("Proportion Missing")
plt.title("Missing Values per Column")
plt.tight_layout()
plt.show()
```



Step 5: Save Raw Data to SQLite Database

Persist the raw DataFrame into a local SQLite database (youtube_trending.db) for SQL-based querying and transformation. Then confirm the data was written by counting the rows.

In [ ]:
```
# Cell 5: Write raw data to SQLite & verify
engine = create_engine('sqlite:///youtube_trending.db')
raw_df.to_sql('raw_data', con=engine, if_exists='replace', index=False)
conn = sqlite3.connect('youtube_trending.db')

# use SQL script to find total rows in raw data
cnt = pd.read_sql_query("SELECT COUNT(*) AS cnt FROM raw_data", conn).iloc[0
print("Rows in raw_data table:", cnt)
```

```
Rows in raw_data table: 40949
```

Pipeline 2 - Data Preprocessing

Step 6: Clean Data with SQL Filtering

Create a cleaned version of the dataset by filtering out videos that:

- Have 0 views
- Were removed or had errors
- Had comments disabled

Then compare row counts before and after cleaning to verify the impact.

In [ ]:
```
# Cell 6: Data cleaning via SQL
c = conn.cursor()
```

```
c.execute("DROP TABLE IF EXISTS cleaned_data")

# 1) Create cleaned_data table with data from raw_data that have views > 0,
c.execute("""
    CREATE TABLE cleaned_data AS
    SELECT *
      FROM raw_data
     WHERE views > 0
       AND video_error_or_removed = 0
       AND comments_disabled = 0
""")
conn.commit()

# 2) Compare total rows of data before and after cleaning
before = pd.read_sql_query("SELECT COUNT(*) AS cnt FROM raw_data", conn).ilo
after  = pd.read_sql_query("SELECT COUNT(*) AS cnt FROM cleaned_data", conn)
print(f"Before cleaning: {before} rows\nAfter cleaning:  {after} rows")
```

```
Before cleaning: 40949 rows
After cleaning:  40293 rows
```

In [ ]:

```
# 3) Quick overview of cleaned_data (YouTube version)
cleaned_df = pd.read_sql_query("""
    SELECT views, likes, dislikes, comment_count
      FROM cleaned_data
     LIMIT 100
""", conn)
print("Sample of cleaned_data:")
display(cleaned_df)

# 4) Numeric summary of key columns
stats = pd.read_sql_query("""
    SELECT
      AVG(views)         AS avg_views,
      AVG(likes)         AS avg_likes,
      AVG(dislikes)      AS avg_dislikes,
      AVG(comment_count) AS avg_comments,
      MIN(views)         AS min_views,
      MAX(views)         AS max_views
    FROM cleaned_data
""", conn)
print("Numeric summary of cleaned_data:")
display(stats)
```

```
Sample of cleaned_data:
```

	views	likes	dislikes	comment_count
0	748374	57527	2966	15954
1	2418783	97185	6146	12703
2	3191434	146033	5339	8181
3	343168	10172	666	2146
4	2095731	132235	1989	17518
...	...	...	...	...
95	836544	40195	373	976
96	284666	16396	81	949
97	3371669	202676	3394	20086
98	195685	14338	171	1070
99	1098897	43875	1326	4702

100 rows × 4 columns

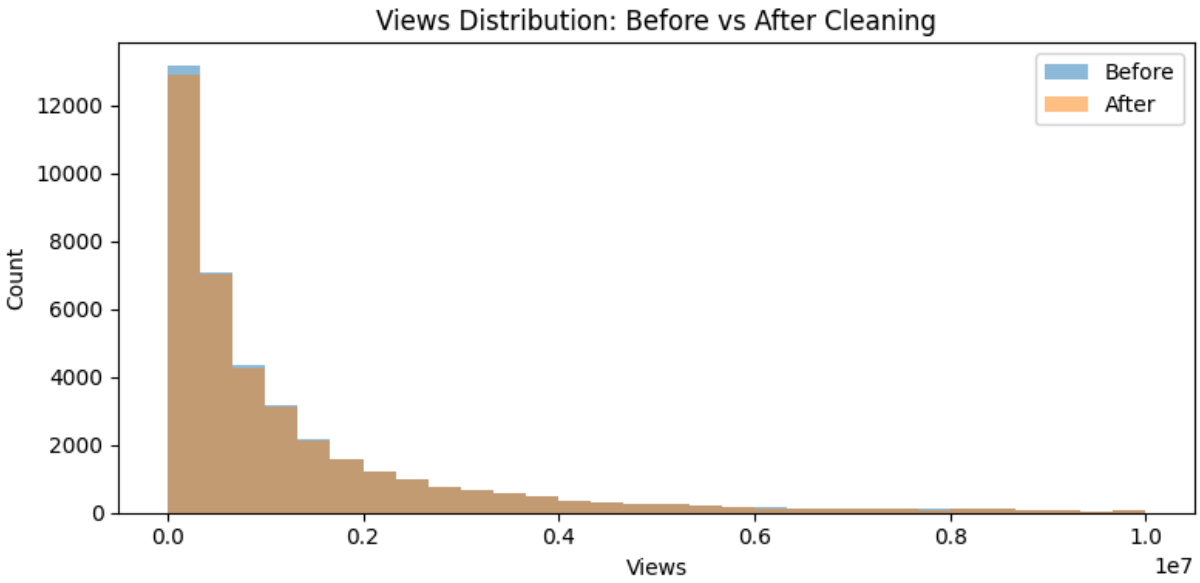
Numeric summary of cleaned_data:

	avg_views	avg_likes	avg_dislikes	avg_comments	min_views	max_vie
0	2.358705e+06	75111.07778	3728.823493	8582.859554	549	2252119

Step 7: Compare Views Distribution Before & After Cleaning

Plot histograms of video view counts before and after data cleaning. This helps assess how the filtering affected the overall distribution and whether extreme or invalid values were removed.

In [ ]:
```
# Cell 7: Views distribution before vs after cleaning (simple histograms)

# 1) Load the 'views' column from the raw and cleaned datasets
before_v = pd.read_sql_query("SELECT views FROM raw_data", conn)['views']
after_v  = pd.read_sql_query("SELECT views FROM cleaned_data", conn)['views'

# 2) Plot histograms to compare the distribution of views before and after c
plt.figure(figsize=(8,4))
plt.hist(before_v, bins=30, alpha=0.5, label='Before', range=(0, 1e7))
plt.hist(after_v,  bins=30, alpha=0.5, label='After',  range=(0, 1e7))
plt.xlabel("Views")
plt.ylabel("Count")
plt.title("Views Distribution: Before vs After Cleaning")
plt.legend()
plt.tight_layout()
plt.show()
```



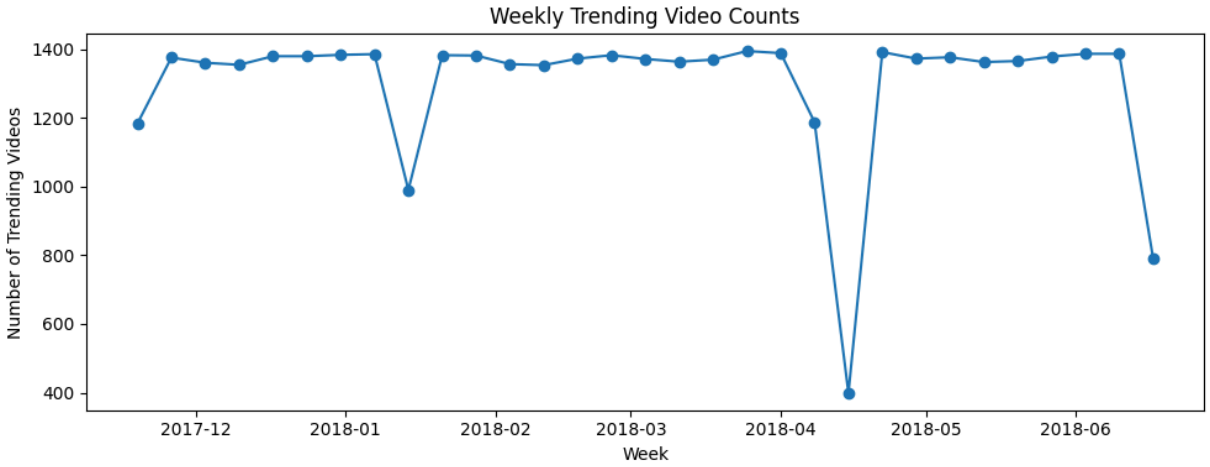
Pipeline 3 - Data Exploration

Step 8: Weekly Trending Video Counts

Use SQL and pandas to parse and convert trending dates. Aggregate video counts per week and plot the number of trending videos over time to uncover seasonality or platform behavior trends.

In [ ]:
```
# Cell 8: Weekly trending video counts (fixed date parsing, simple line plot

# 1) Query daily trending video counts with corrected date format
monthly = pd.read_sql_query("""
    SELECT
      date(
        '20' || substr(trending_date,1,2) || '-' ||
        substr(trending_date,7,2)       || '-' ||
        substr(trending_date,4,2)
      ) AS TrendDate,
      COUNT(*) AS Count
    FROM cleaned_data
    GROUP BY TrendDate
    ORDER BY TrendDate
""", conn)

# 2) Ensure TrendDate is a proper datetime type
monthly['TrendDate'] = pd.to_datetime(monthly['TrendDate'], format='%Y-%m-%c

# 3) Resample counts by week, summing daily counts
weekly_counts = monthly.set_index('TrendDate')['Count'] \
                       .resample('W') \
                       .sum()

# 4) Plot weekly trending video counts with a simple line plot
```

```
plt.figure(figsize=(10,4))
plt.plot(weekly_counts.index, weekly_counts.values, marker='o')
plt.xlabel("Week")
plt.ylabel("Number of Trending Videos")
plt.title("Weekly Trending Video Counts")
plt.tight_layout()
plt.show()

# 5) Display the monthly daily counts dataframe
display(monthly)
```



	TrendDate	Count
0	2017-11-14	198
1	2017-11-15	198
2	2017-11-16	197
3	2017-11-17	197
4	2017-11-18	197
...	...	...
200	2018-06-10	198
201	2018-06-11	198
202	2018-06-12	198
203	2018-06-13	197
204	2018-06-14	198

205 rows × 2 columns

Pipeline 4 - Feature Engineering

Step 9: Engagement & Popularity

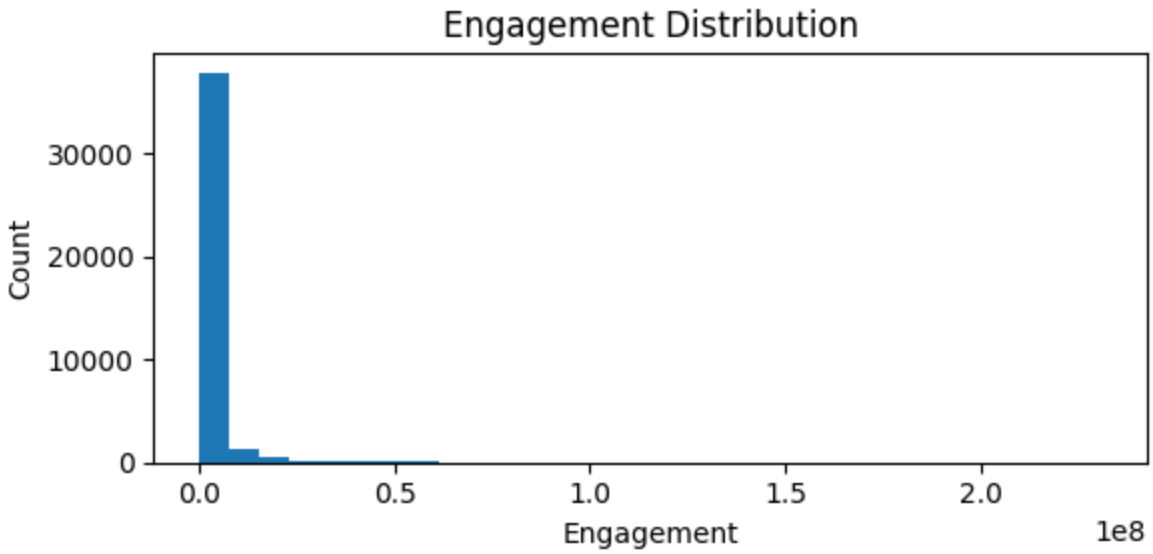
Add basic new features:

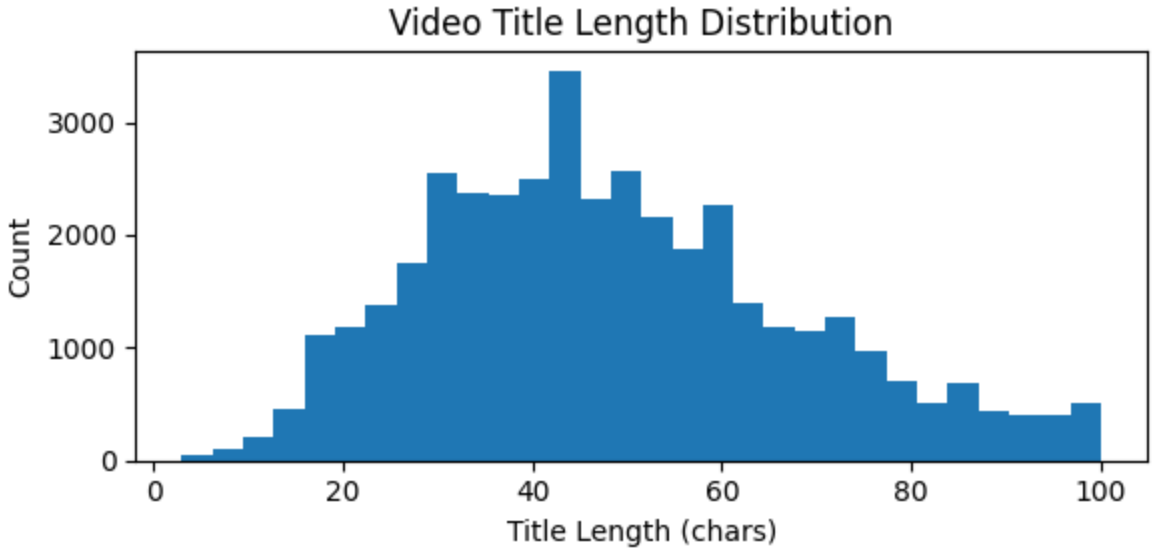
- **Engagement**: Combined count of views, likes, and comments
- **TitleLength**: Character count of the video title
- **is_popular**: A binary label marking videos with above-median views as "popular"

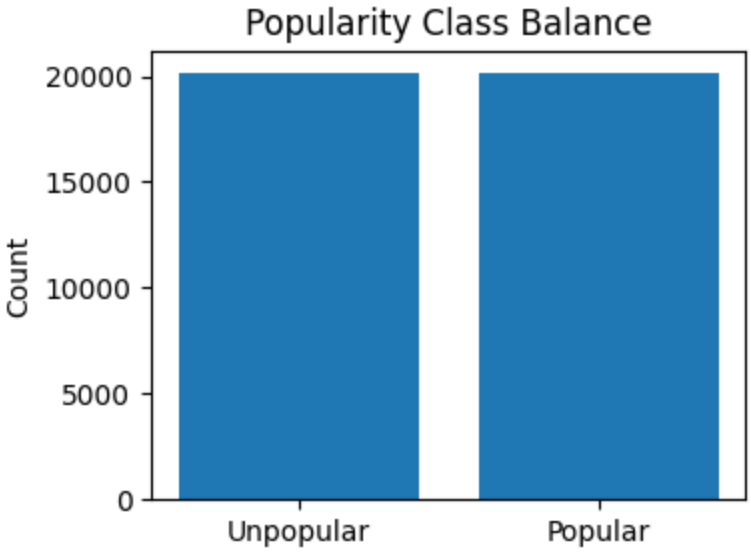
Also plot distributions of these new features and visualize the class balance of popularity.

In [ ]:
```
# Cell 9: Feature Engineering — Basic numeric & title length
# ------------------------------------------------------------
# 1) Compute median views from cleaned_data
median_views = pd.read_sql_query("SELECT views FROM cleaned_data", conn)['vi

# 2) Calculate and create new features: total engagement (sum of views + lik
c.execute("DROP TABLE IF EXISTS feat_basic")
c.execute(f"""
    CREATE TABLE feat_basic AS
    SELECT *,
           (views + likes + comment_count)         AS Engagement,
           LENGTH(title)                          AS TitleLength,
           CASE WHEN views > {median_views:.0f} THEN 1 ELSE 0 END AS is_popu
      FROM cleaned_data
""")
conn.commit()

# 3) Load new features from database
fb = pd.read_sql_query("SELECT Engagement, TitleLength, is_popular FROM feat

# 4) Plot Engagement distribution
plt.figure(figsize=(6,3))
plt.hist(fb['Engagement'], bins=30)
plt.title("Engagement Distribution")
plt.xlabel("Engagement")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 5) Plot Title Length distribution
plt.figure(figsize=(6,3))
plt.hist(fb['TitleLength'], bins=30)
plt.title("Video Title Length Distribution")
plt.xlabel("Title Length (chars)")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 6) Plot Popular vs Unpopular count
plt.figure(figsize=(4,3))
counts = fb['is_popular'].value_counts().sort_index()
plt.bar(['Unpopular','Popular'], counts.values)
plt.title("Popularity Class Balance")
plt.ylabel("Count")
```

```
plt.tight_layout()
plt.show()

# 7) Display the feature table
display(fb)
```







	Engagement	TitleLength	is_popular
0	821855	34	1
1	2528671	62	1
2	3345648	53	1
3	355486	32	0
4	2245484	24	1
...	...	...	...
40288	1726426	28	1
40289	1128742	26	1
40290	1118511	84	1
40291	5866858	35	1
40292	10807993	64	1

40293 rows × 3 columns

Step 10: Ratio-Based Metrics

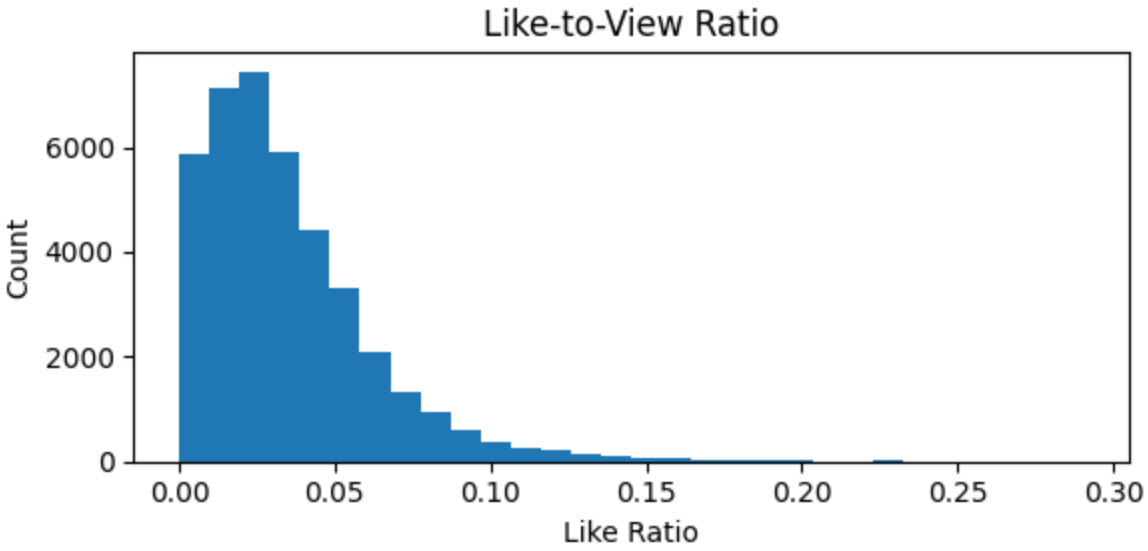
Add two derived metrics:

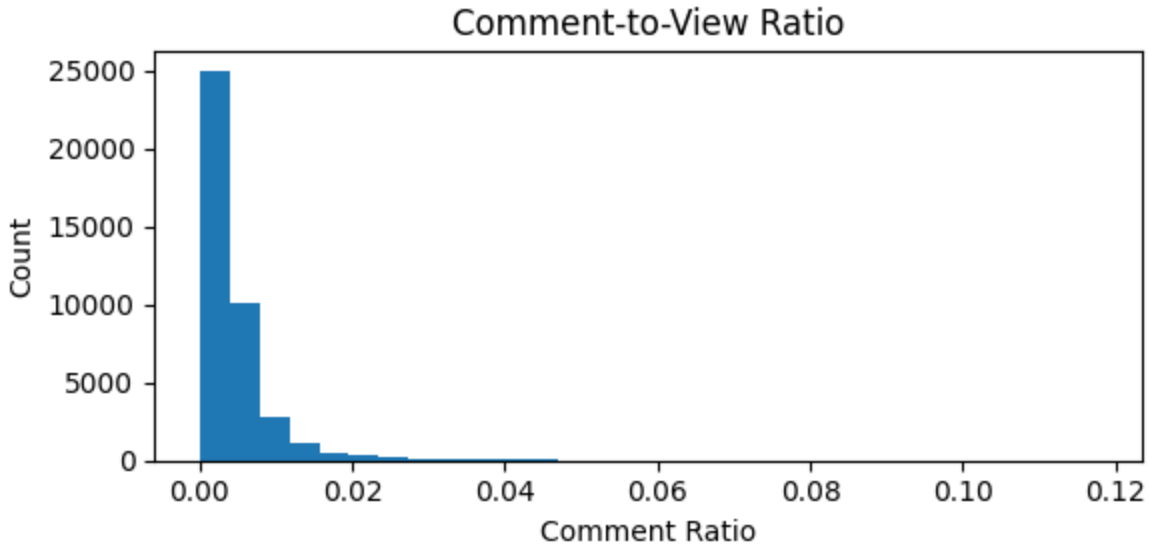
- **like_ratio**: Likes per view
- **comment_ratio**: Comments per view

These ratios normalize engagement by view count. Histograms show their distributions, giving insight into relative engagement.

In [ ]:
```
# Cell 10: Feature Engineering – Derived ratios
# ---------------------------------------------
```

```
# 1) Create ratio features: like-to-view and comment-to-view ratios from bas
c.execute("DROP TABLE IF EXISTS feat_ratios")
c.execute("""
    CREATE TABLE feat_ratios AS
    SELECT *,
           CAST(likes AS FLOAT)/views        AS like_ratio,
           CAST(comment_count AS FLOAT)/views AS comment_ratio
      FROM feat_basic
""")
conn.commit()

# 2) Load the ratio features from the database
fr = pd.read_sql_query("SELECT like_ratio, comment_ratio FROM feat_ratios",

# 3) Plot Like-to-View Ratio distribution as histogram
plt.figure(figsize=(6,3))
plt.hist(fr['like_ratio'], bins=30)
plt.title("Like-to-View Ratio")
plt.xlabel("Like Ratio")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 4) Plot Comment-to-View Ratio distribution as histogram
plt.figure(figsize=(6,3))
plt.hist(fr['comment_ratio'], bins=30)
plt.title("Comment-to-View Ratio")
plt.xlabel("Comment Ratio")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 5) Display the ratio features table
display(fr)
```



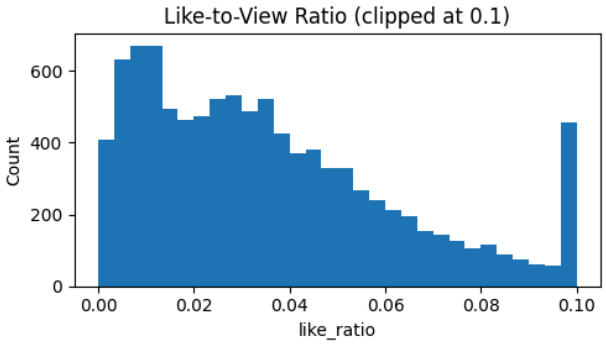


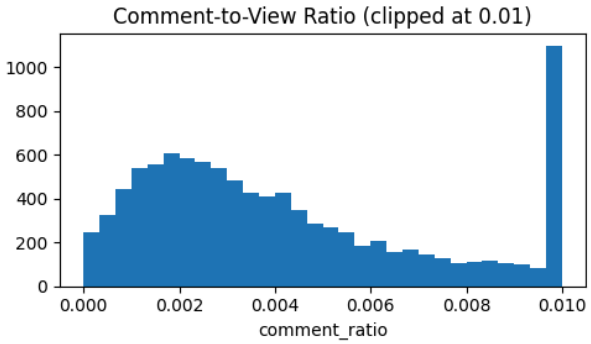
	like_ratio	comment_ratio
0	0.076869	0.021318
1	0.040179	0.005252
2	0.045758	0.002563
3	0.029641	0.006253
4	0.063097	0.008359
...	...	...
40288	0.022639	0.001576
40289	0.056356	0.003696
40290	0.045073	0.003743
40291	0.034086	0.002312
40292	0.034647	0.014049

40293 rows × 2 columns

In [ ]:
```
# New Cell: Histogram of the new ratio features
# Load sample (10000 rows) of ratio features for analysis and visualization
ratios_df = pd.read_sql_query("""
    SELECT like_ratio, comment_ratio
      FROM feat_ratios
    LIMIT 10000
""", conn)

# Plot histogram of like-to-view ratio (values clipped at 0.1 for better vis
plt.figure(figsize=(10,3))
plt.subplot(1,2,1)
plt.hist(ratios_df['like_ratio'].clip(upper=0.1), bins=30)
plt.title("Like-to-View Ratio (clipped at 0.1)")
plt.xlabel("like_ratio")
plt.ylabel("Count")
```

```
# Plot histogram of comment-to-view ratio (values clipped at 0.01 for better
plt.subplot(1,2,2)
plt.hist(ratios_df['comment_ratio'].clip(upper=0.01), bins=30)
plt.title("Comment-to-View Ratio (clipped at 0.01)")
plt.xlabel("comment_ratio")
plt.tight_layout()
plt.show()

# Show the SQL table so far
display(ratios_df)
```





	like_ratio	comment_ratio
0	0.076869	0.021318
1	0.040179	0.005252
2	0.045758	0.002563
3	0.029641	0.006253
4	0.063097	0.008359
...	...	...
9995	0.011471	0.000595
9996	0.030624	0.001034
9997	0.006333	0.001079
9998	0.010888	0.001472
9999	0.013744	0.004686

10000 rows × 2 columns

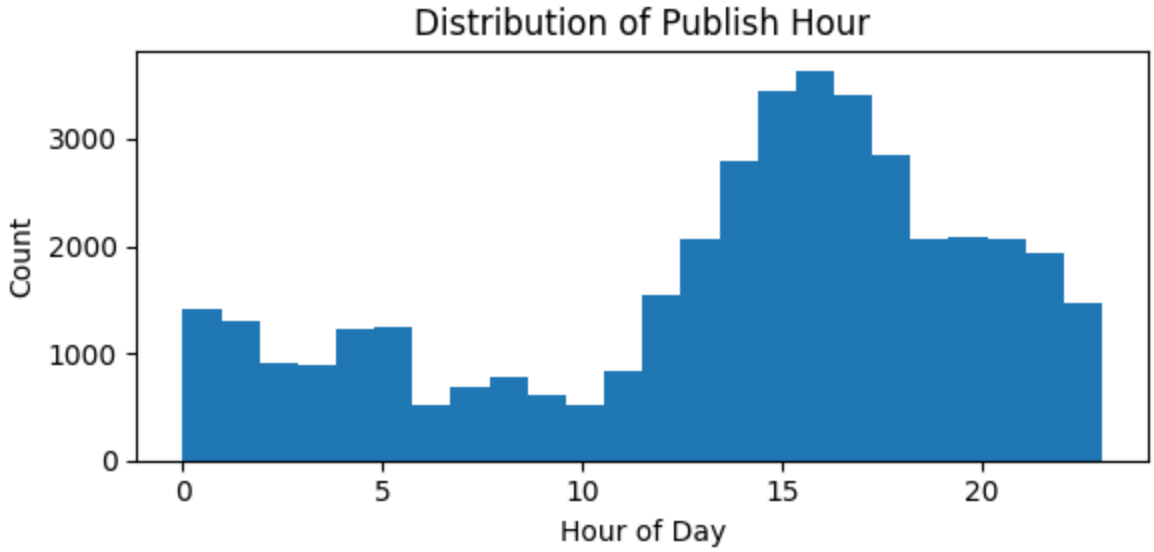
Step 11: Temporal Attributes

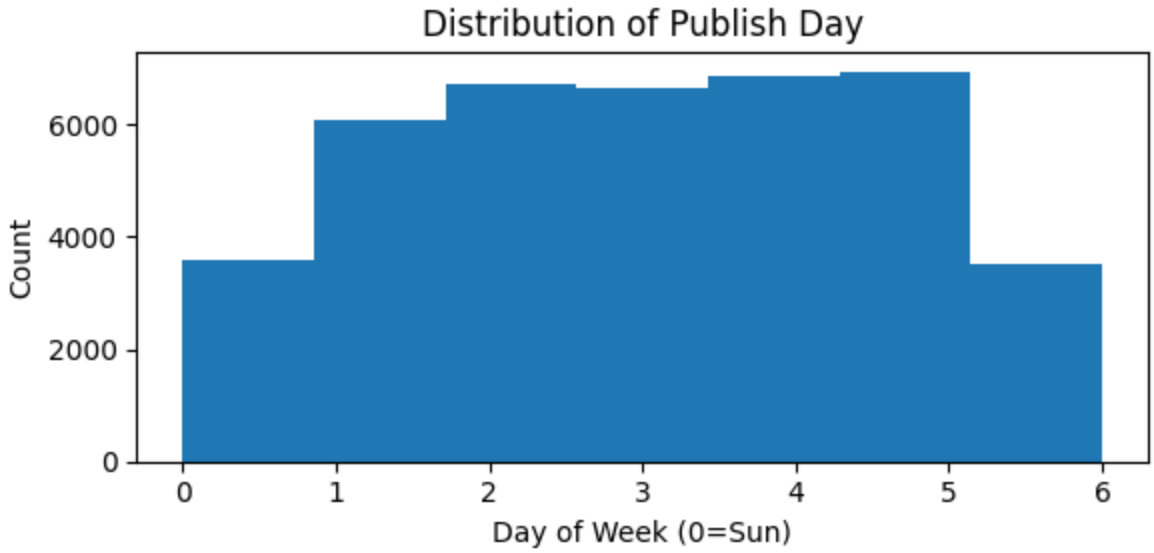
Extract:

- **publish_hour**: Hour of video publishing
- **publish_dayofweek**: Day of the week (0 = Sunday)

Plot these features to understand when videos are commonly uploaded.

In [ ]:
```
# Cell 11: Feature Engineering — Temporal features
# ------------------------------------------------

# 1) Extract publish hour and publish day of week from publish_time, creatin
c.execute("DROP TABLE IF EXISTS feat_temp")
c.execute("""
    CREATE TABLE feat_temp AS
    SELECT f.*,
           CAST(strftime('%H', publish_time) AS INTEGER) AS publish_hour,
           CAST(strftime('%w', publish_time) AS INTEGER) AS publish_dayofwee
      FROM feat_ratios f
""")
conn.commit()

# 2) Load the new temporal features from the database
ft = pd.read_sql_query("SELECT publish_hour, publish_dayofweek FROM feat_tem

# 3) Plot Publish Hour distribution as histogram
plt.figure(figsize=(6,3))
plt.hist(ft['publish_hour'], bins=24, range=(0,23))
plt.title("Distribution of Publish Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 4) Plot Publish Day of Week distribution as histogram
plt.figure(figsize=(6,3))
plt.hist(ft['publish_dayofweek'], bins=7, range=(0,6))
plt.title("Distribution of Publish Day")
plt.xlabel("Day of Week (0=Sun)")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 5) Display the extracted temporal features table
display(ft)
```


Distribution of Publish Hour
Count
3000
2000
1000
0
0
5
10
15
20
Hour of Day


Distribution of Publish Day
Count
6000
4000
2000
0
0
1
2
3
4
5
6
Day of Week (0=Sun)

	publish_hour	publish_dayofweek
0	17	1
1	7	1
2	19	0
3	11	1
4	18	0
...	...	...
40288	13	5
40289	1	5
40290	17	5
40291	17	4
40292	17	4

40293 rows × 2 columns

Step 12: Channel-Level Aggregates

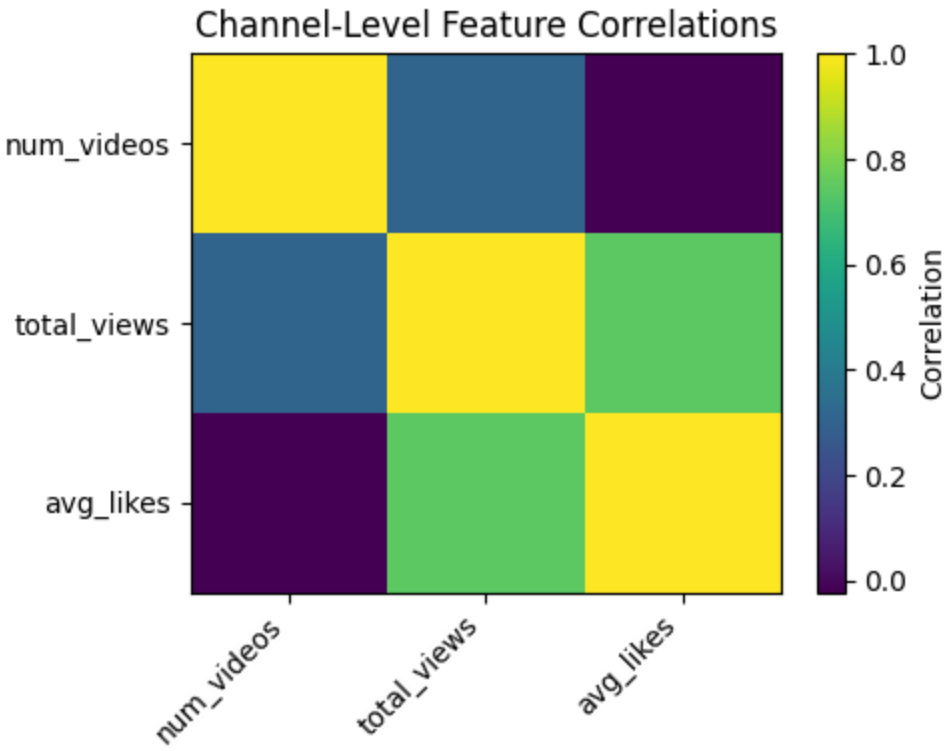
Aggregate features at the channel level:

- Number of videos
- Total views
- Average likes

Then merge these channel-level metrics back into the main dataset. Finally, show correlations between all numeric features using a heatmap to check for redundancy or predictive power.

In [ ]:
```
# Cell 12: Feature Engineering – Channel aggregates & correlation
# ----------------------------------------------------------------

# 1) Aggregate channel-level stats: count of videos, total views, and averag
c.execute("DROP TABLE IF EXISTS channel_agg")
c.execute("""
    CREATE TABLE channel_agg AS
    SELECT channel_title,
           COUNT(*)    AS num_videos,
           SUM(views)  AS total_views,
           AVG(likes)  AS avg_likes
      FROM feat_temp
  GROUP BY channel_title
""")
conn.commit()

# 2) Combine channel-level aggregates with detailed features by joining on c
```

```
c.execute("DROP TABLE IF EXISTS features_final")
c.execute("""
    CREATE TABLE features_final AS
    SELECT t.*, c.num_videos, c.total_views, c.avg_likes
      FROM feat_temp t
 LEFT JOIN channel_agg c USING(channel_title)
""")
conn.commit()

# 3) Load selected features from final table and compute their correlation m
ff = pd.read_sql_query("SELECT num_videos, total_views, avg_likes FROM featu
corr = ff.corr()

# 4) Plot Correlation Matrix with Heatmap
plt.figure(figsize=(5,4))
plt.imshow(corr, cmap='viridis', aspect='auto')
plt.colorbar(label='Correlation')
plt.xticks(range(len(corr)), corr.columns, rotation=45, ha='right')
plt.yticks(range(len(corr)), corr.columns)
plt.title("Channel-Level Feature Correlations")
plt.tight_layout()
plt.show()

# 5) Show the SQL table so far
display(ff)
```



	num_videos	total_views	avg_likes
0	95	232745266	108499.389474
1	24	97556377	116435.375000
2	74	240999117	175268.621622
3	147	142231380	20405.380952
4	89	590616191	330282.831461
...	...	...	...
40288	65	57641422	24093.476923
40289	50	53189544	72969.140000
40290	36	28093537	35714.750000
40291	3	16880390	192520.666667
40292	41	315404711	281794.975610

40293 rows × 3 columns

Pipeline 5 - Model Selection

Step 13: Prepare Data for Engagement Prediction & Visual EDA

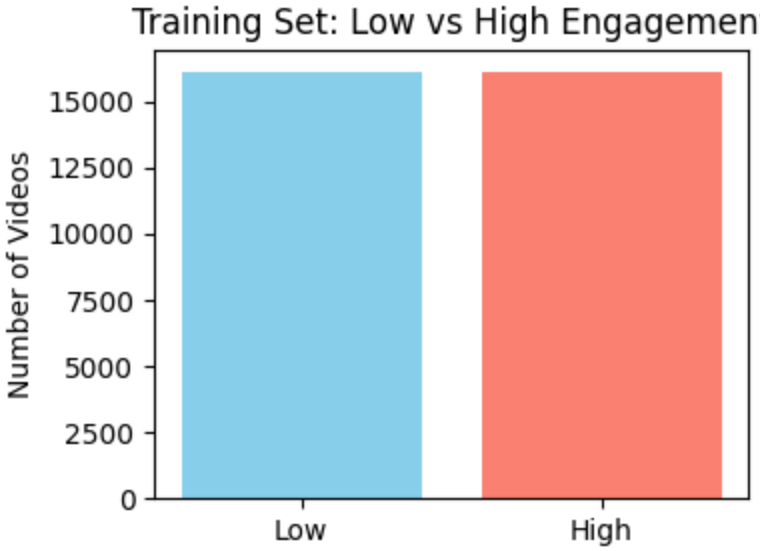
Define a new binary prediction task: classify videos into high vs low engagement (based on median `Engagement`).

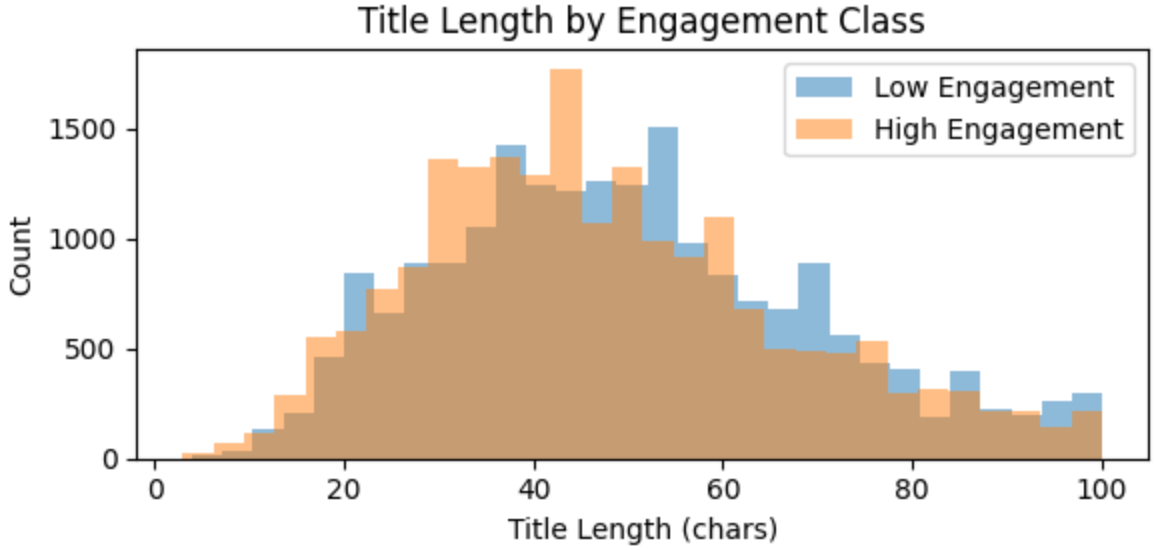
In [ ]:
```
# Cell 13: Prepare Data for Engagement-Level Prediction & Extended EDA
# ----------------------------------------------------------------------
# 1) Load our engineered feature table
df_feat = pd.read_sql_query("SELECT * FROM features_final", conn)

# 2) Compute median Engagement (views + likes + comments)
median_eng = df_feat['Engagement'].median()
print(f"Median Engagement: {median_eng:.0f}")

# 3) Define binary target: high engagement if above median
df_feat['high_engagement'] = (df_feat['Engagement'] > median_eng).astype(int

# 4) Select input features (omit raw counts to avoid leakage)
features = [
    'TitleLength','like_ratio','comment_ratio',
    'publish_hour','publish_dayofweek',
    'num_videos','total_views','avg_likes'
]
X = df_feat[features]
```

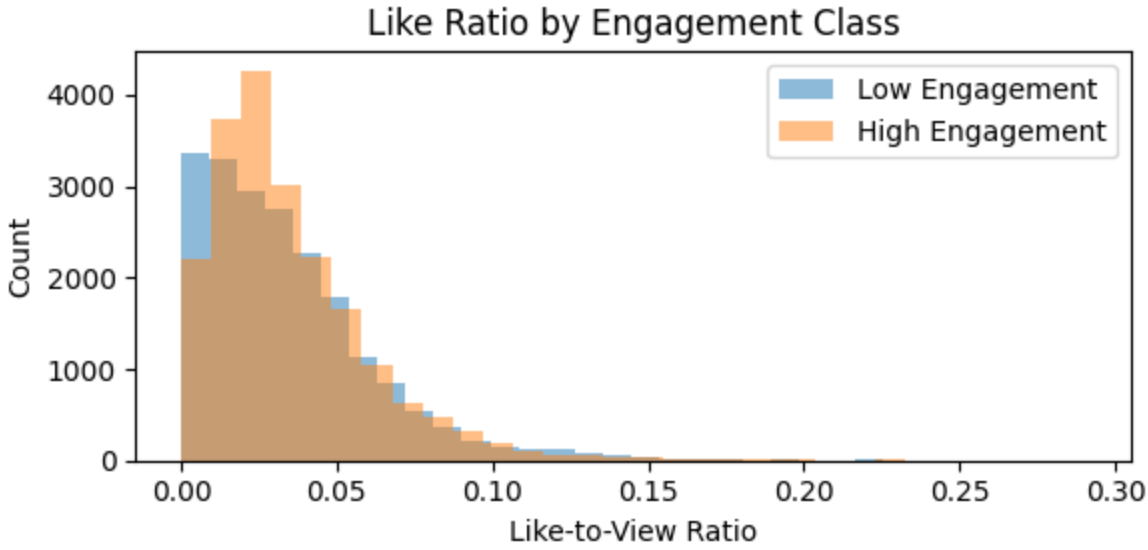
```
y = df_feat['high_engagement']

# 5) Train/test split (stratify to keep class balance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Train shape:", X_train.shape, "Test shape:", X_test.shape)

# --- Extended Visualizations ---

# A) Class balance in training set
plt.figure(figsize=(4,3))
counts = y_train.value_counts().sort_index()
plt.bar(['Low','High'], counts.values, color=['skyblue','salmon'])
plt.title("Training Set: Low vs High Engagement")
plt.ylabel("Number of Videos")
plt.tight_layout()
plt.show()

# B) Title length distribution by engagement class
plt.figure(figsize=(6,3))
plt.hist(df_feat.loc[df_feat.high_engagement==0, 'TitleLength'],
         bins=30, alpha=0.5, label='Low Engagement')
plt.hist(df_feat.loc[df_feat.high_engagement==1, 'TitleLength'],
         bins=30, alpha=0.5, label='High Engagement')
plt.xlabel("Title Length (chars)")
plt.ylabel("Count")
plt.title("Title Length by Engagement Class")
plt.legend()
plt.tight_layout()
plt.show()

# C) Like-to-view ratio by engagement class
plt.figure(figsize=(6,3))
plt.hist(df_feat.loc[df_feat.high_engagement==0, 'like_ratio'],
         bins=30, alpha=0.5, label='Low Engagement')
plt.hist(df_feat.loc[df_feat.high_engagement==1, 'like_ratio'],
         bins=30, alpha=0.5, label='High Engagement')
plt.xlabel("Like-to-View Ratio")
plt.ylabel("Count")
plt.title("Like Ratio by Engagement Class")
plt.legend()
plt.tight_layout()
plt.show()

# D) Comment-to-view ratio by engagement class
plt.figure(figsize=(6,3))
plt.hist(df_feat.loc[df_feat.high_engagement==0, 'comment_ratio'],
         bins=30, alpha=0.5, label='Low Engagement')
plt.hist(df_feat.loc[df_feat.high_engagement==1, 'comment_ratio'],
         bins=30, alpha=0.5, label='High Engagement')
plt.xlabel("Comment-to-View Ratio")
plt.ylabel("Count")
plt.title("Comment Ratio by Engagement Class")
plt.legend()
```

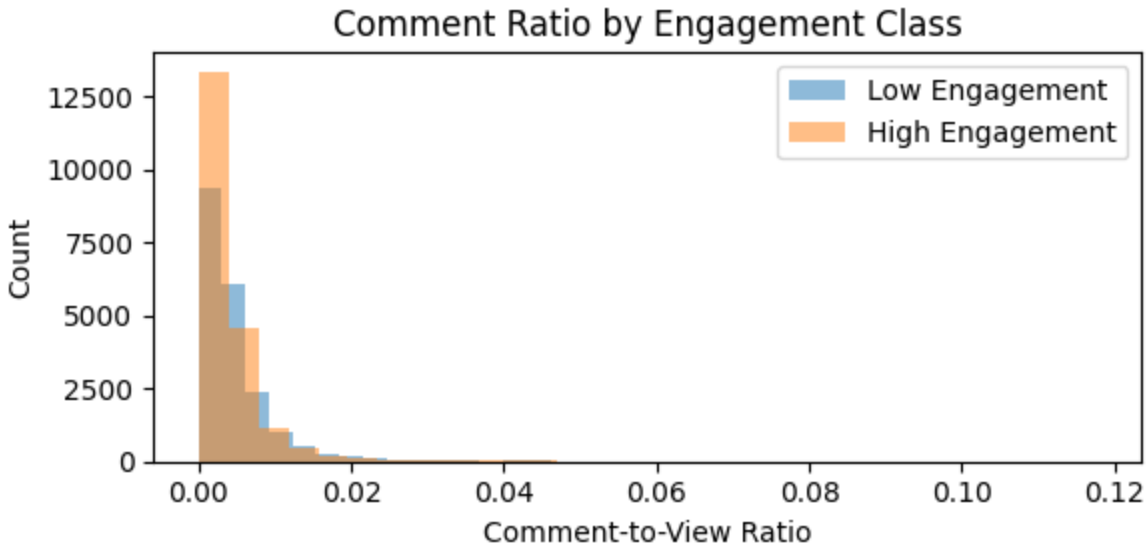
```
plt.tight_layout()
plt.show()
```

```
Median Engagement: 706995
Train shape: (32234, 8) Test shape: (8059, 8)
```









Pipeline 6 - Hyperparameter Tuning & Pipeline 7 - Model Training

Step 14: Train & Tune Models Quickly Using GridSearchCV

Train three ensemble classifiers (Random Forest, Gradient Boosting, AdaBoost) with small grids and fewer CV folds for speed. Store best models and their test accuracies.

In [ ]:
```
# Cell 14: Faster Hyperparameter Tuning & Evaluation
# ------------------------------------------------
# To cut training time, we:
```

```
#  - Reduce CV folds from 5 → 3
#  - Narrow each model's grid to one or two settings
#  - Keep n_jobs=-1 so each search still uses all cores efficiently

from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

# 1) Define our ensemble models
models = {
    'RandomForest':    RandomForestClassifier(random_state=42),
    'GradientBoosting':GradientBoostingClassifier(random_state=42),
    'AdaBoost':        AdaBoostClassifier(random_state=42)
}

# 2) Slimmed-down grids for quick tuning
param_grid = {
    'RandomForest': {
        'n_estimators': [100],    # single setting
        'max_depth':    [None, 10]
    },
    'GradientBoosting': {
        'n_estimators':   [100],
        'learning_rate':  [0.1]   # single setting
    },
    'AdaBoost': {
        'n_estimators':  [50],    # single setting
        'learning_rate': [1.0]
    }
}

tuned_acc = {}
tuned_models = {}

# 3) Run 3-fold CV for each model
for name, model in models.items():
    print(f"Tuning {name} (fast)…")
    gs = GridSearchCV(
        estimator=model,
        param_grid=param_grid[name],
        cv=3,              # fewer folds
        scoring='accuracy',
        n_jobs=-1
    )
    gs.fit(X_train, y_train)    # much faster grid search
    best = gs.best_estimator_
    tuned_models[name] = best

    # 4) Evaluate on test set
    preds = best.predict(X_test)
    tuned_acc[name] = accuracy_score(y_test, preds)
    print(f"→ {name} best params: {gs.best_params_}, test accuracy: {tuned_a
```

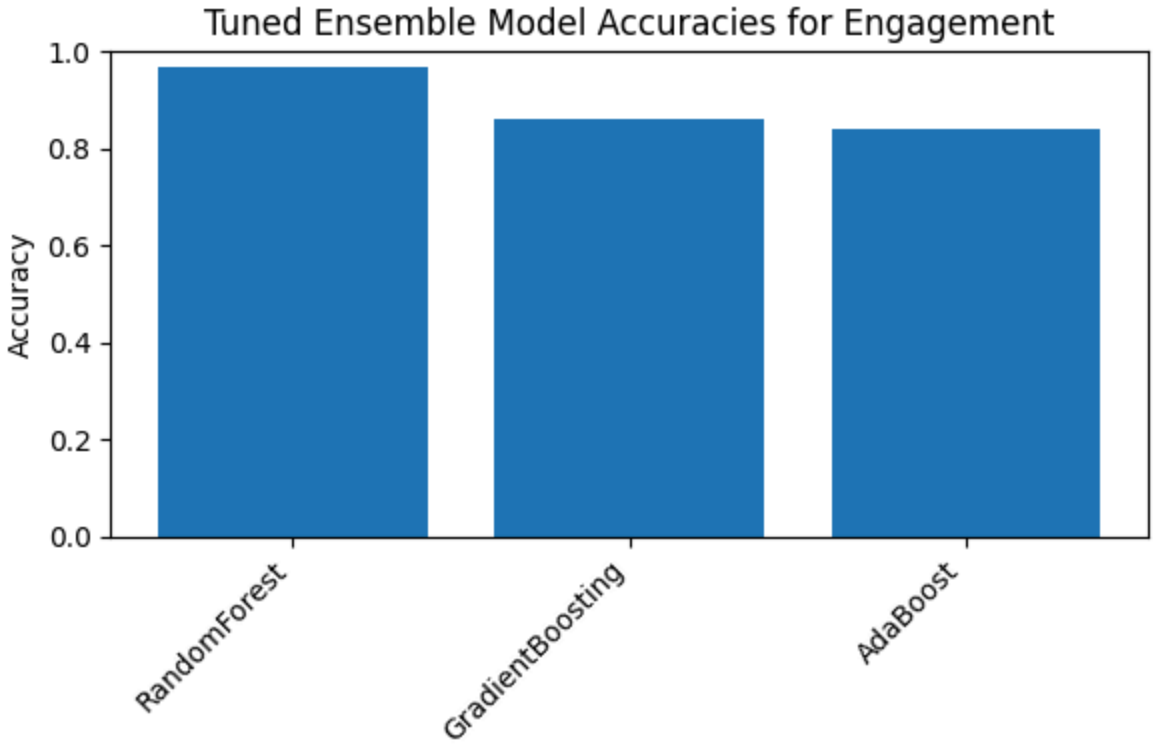
```
Tuning RandomForest (fast)…
→ RandomForest best params: {'max_depth': None, 'n_estimators': 100}, test a
ccuracy: 0.9666
Tuning GradientBoosting (fast)…
→ GradientBoosting best params: {'learning_rate': 0.1, 'n_estimators': 100},
test accuracy: 0.8623
Tuning AdaBoost (fast)…
→ AdaBoost best params: {'learning_rate': 1.0, 'n_estimators': 50}, test acc
uracy: 0.8383
```

Pipeline 7 - Model Evaluation

Step 15: Visualize Model Accuracies

Plot a simple bar chart comparing the tuned test accuracies of the three models. This helps choose the best-performing classifier for dashboard deployment.

In [ ]:
```python
# Cell 15: Plot Tuned Model Accuracies
names      = list(tuned_acc.keys())          # List of model names from tune
accuracies = [tuned_acc[n] for n in names]   # Corresponding accuracy values
x          = np.arange(len(names))           # Array of indices for plotting

# Plot Accuracy of each Tuned Model with Bar Chart
plt.figure(figsize=(6,4))
plt.bar(x, accuracies)
plt.xticks(x, names, rotation=45, ha='right')
plt.ylim(0,1)
plt.ylabel("Accuracy")
plt.title("Tuned Ensemble Model Accuracies for Engagement")
plt.tight_layout()
plt.show()
```



Pipeline 8 - Model Deployment

Step 16: Gradio Dashboard – Insights & Model Outputs

Create an interactive dashboard using Gradio. It shows:

1. Title length distribution by engagement class
2. Tuned model accuracies
3. Top 10 channels by predicted engagement (using Random Forest)
4. Feature importance chart from Random Forest
   The dashboard makes it easy to explore the modeling results and insights visually.

In [ ]:
```
# Cell 16: Gradio Dashboard — Title Length EDA & Extended Top Channels
def show_dashboard():
    # 1) Title length distribution by engagement class
    fig1, ax1 = plt.subplots(figsize=(6,3))
    ax1.hist(df_feat.loc[df_feat.high_engagement==0, 'TitleLength'],
             bins=30, alpha=0.5, label='Low Engagement')
    ax1.hist(df_feat.loc[df_feat.high_engagement==1, 'TitleLength'],
             bins=30, alpha=0.5, label='High Engagement')
    ax1.set_xlabel("Title Length (chars)")
    ax1.set_ylabel("Count")
    ax1.set_title("Title Length by Engagement Class")
    ax1.legend()
```

```
    fig1.tight_layout()

    # 2) Tuned model accuracies
    fig2, ax2 = plt.subplots(figsize=(6,3))
    names_list = list(tuned_acc.keys())
    accs = [tuned_acc[n] for n in names_list]
    x = np.arange(len(names_list))
    ax2.bar(x, accs, color='lightgreen')
    ax2.set_xticks(x)
    ax2.set_xticklabels(names_list, rotation=45, ha='right')
    ax2.set_ylim(0,1)
    ax2.set_ylabel("Accuracy")
    ax2.set_title("Tuned Model Accuracies")
    fig2.tight_layout()

    # 3) Top-10 channels by average predicted engagement using the best Rand
    best_model = tuned_models['RandomForest']
    probs = best_model.predict_proba(X_test)[:,1]
    video_meta = df_feat.loc[X_test.index, ['channel_title']].copy()
    video_meta['pred_high_eng'] = probs
    channel_probs = video_meta.groupby('channel_title')['pred_high_eng'].mea
    top10_channels = (
        channel_probs
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
        .rename(columns={'pred_high_eng': 'avg_pred_high_eng'})
    )

    # 4) Feature importances of the RandomForest
    importances = best_model.feature_importances_
    imp_df = pd.Series(importances, index=features).sort_values()
    fig4, ax4 = plt.subplots(figsize=(6,3))
    imp_df.plot.barh(ax=ax4)
    ax4.set_title("RandomForest Feature Importances")
    fig4.tight_layout()

    # Return: title-length hist, accuracies bar, top10 DataFrame, importance
    return fig1, fig2, top10_channels, fig4

gr.Interface(
    fn=show_dashboard,
    inputs=[],
    outputs=[
        gr.Plot(label="Title Length by Engagement Class"),
        gr.Plot(label="Tuned Model Accuracies"),
        gr.Dataframe(label="Top 10 Channels by Predicted Engagement"),
        gr.Plot(label="Feature Importances")
    ],
    title="US YouTube Video Engagement Dashboard & Model Insights",
    description="Title length EDA, tuned model performance, top channels, ar
).launch()
```

```
Setting queue=True in a Colab notebook requires sharing enabled. Setting `sh
are=True` (you can turn this off by setting `share=False` in `launch()` expl
icitly).

Colab notebook detected. To show errors in colab notebook, set debug=True in
launch()
Running on public URL: https://817a6f93cbdcf32aa9.gradio.live

This share link expires in 72 hours. For free permanent hosting and GPU upgr
ades, run `gradio deploy` from Terminal to deploy to Spaces (https://hugging
face.co/spaces)
```

No interface is running right now

Out[ ]: